

Introduction to ResNet Architecture

Contents

- Motivation for Residual Network
- Vanishing Gradient problem
- Introduction to Residual Blocks and its types
- ResNet Architecture

ResNet 34

ResNet 50

Motivation & Historical Context

- ✓ To solve a complex problem, additional layers are stacked in the Deep Neural Networks which results in improved accuracy and performance.
- ✓ The intuition behind adding more layers is that these layers progressively learn more complex features.

But, can we keep increasing the layers indefinitely for better accuracy?

- ✓ It was observed that deep networks are hard to train because of the notorious **vanishing gradient problem**

Vanishing Gradient Problem

Why it happens?

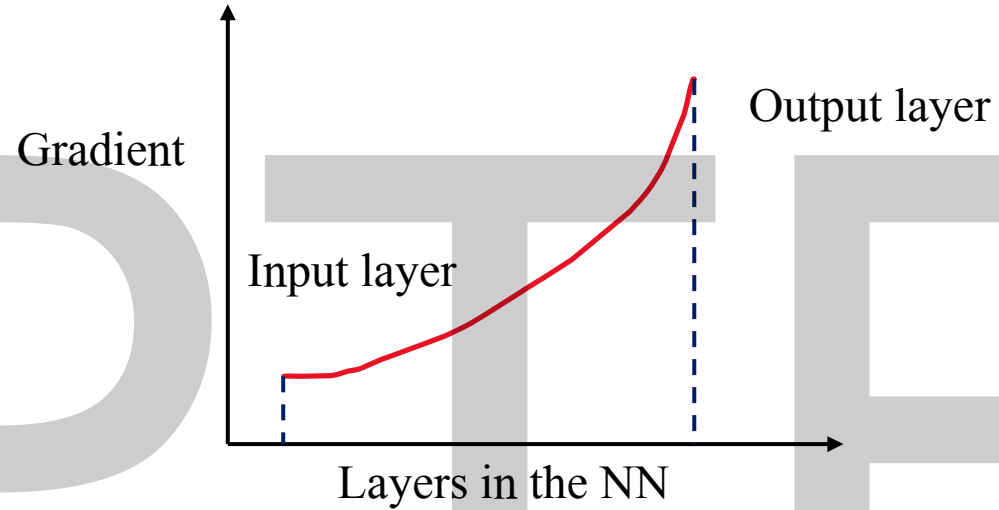
- ✓ Vanishing gradient happens as activation functions like sigmoid and tanh squash values to a small range (0 to 1) and (-1 to 1), so derivatives become tiny.

Sigmoid activation function

Sigmoid outputs values in the range (0-1)

Derivative of sigmoid:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$



Tanh Activation function:

Outputs values in the range -1 to 1 .

The derivative of tanh:

$$\tanh'(x) = 1 - \tanh(x)^2$$

- ✓ During backpropagation, gradients are multiplied through each layer of the network
- ✓ Successive multiplication through each layer makes them very small.

Why it is a problem

Earlier layers stop learning as their weights hardly change because gradients are near zero.

Solution for Vanishing Gradient Problem

- Using Non-Saturating Activation Functions like ReLU (Rectified Linear Unit) or leaky ReLU, which do not squash the input values into a narrow range and thus help solving the problem of vanishing gradient as done in Alexnet.
- GoogLeNet used Auxiliary classifiers for mitigating vanishing gradient problem.
- **Residual Connections:**
These are shortcut/skip connections that bypass certain layers, reducing the number of multiplications and helping maintain the gradient size.

Introduced by Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun in 2016 in the paper titled
'Deep Residual Learning for Image Recognition'(Microsoft Research)
<https://doi.org/10.1109/CVPR.2016.90>

Residual Block

The skip connection connects activations of a layer to further layers by skipping some layers in between.

In a normal deep network, each layer learns a mapping:

$$H(x)$$

where x is the input, and $H(x)$ is the desired output.

ResNet changes this to learning a residual function:

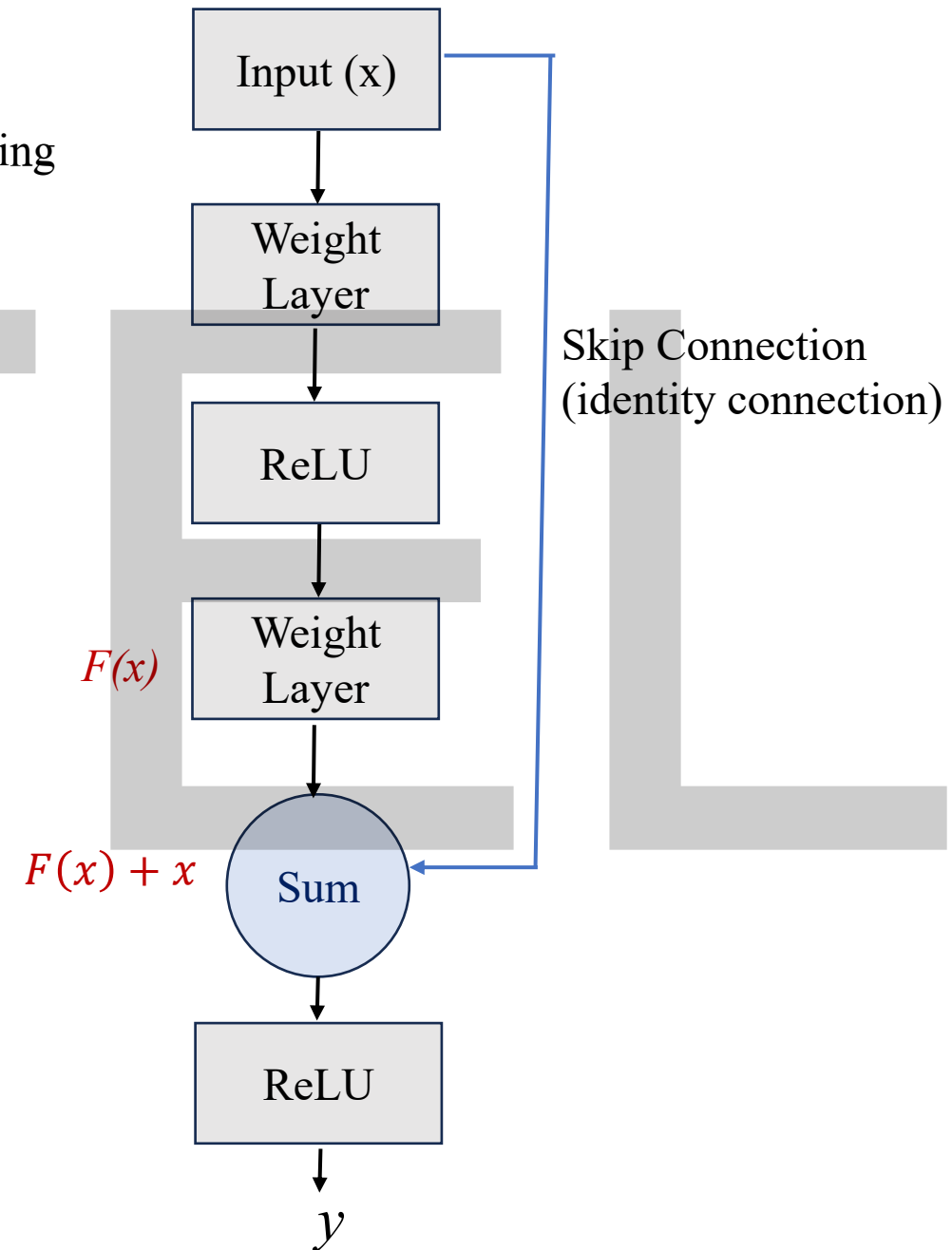
$$F(x) = H(x) - x$$

or equivalently,

$$H(x) = F(x) + x$$

$$y = \text{ReLU}(F(x) + x)$$

DONE WHEN INPUT AND OUTPUT DIMENSIONS ARE SAME



How Residual Block Mitigates the Vanishing Gradient Problem

Let's compute the gradient of the loss L w.r.t. x :

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Now, since $y = F(x) + x$:

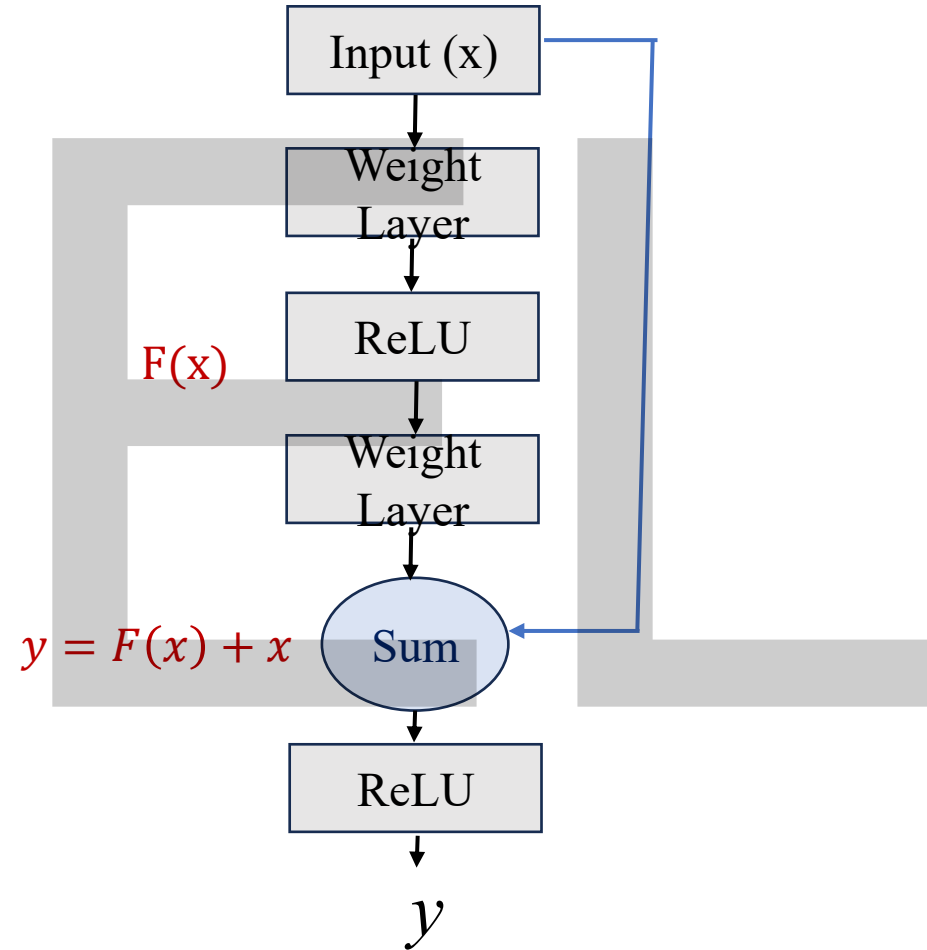
$$\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + I$$

So:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \left(I + \frac{\partial F(x)}{\partial x} \right)$$

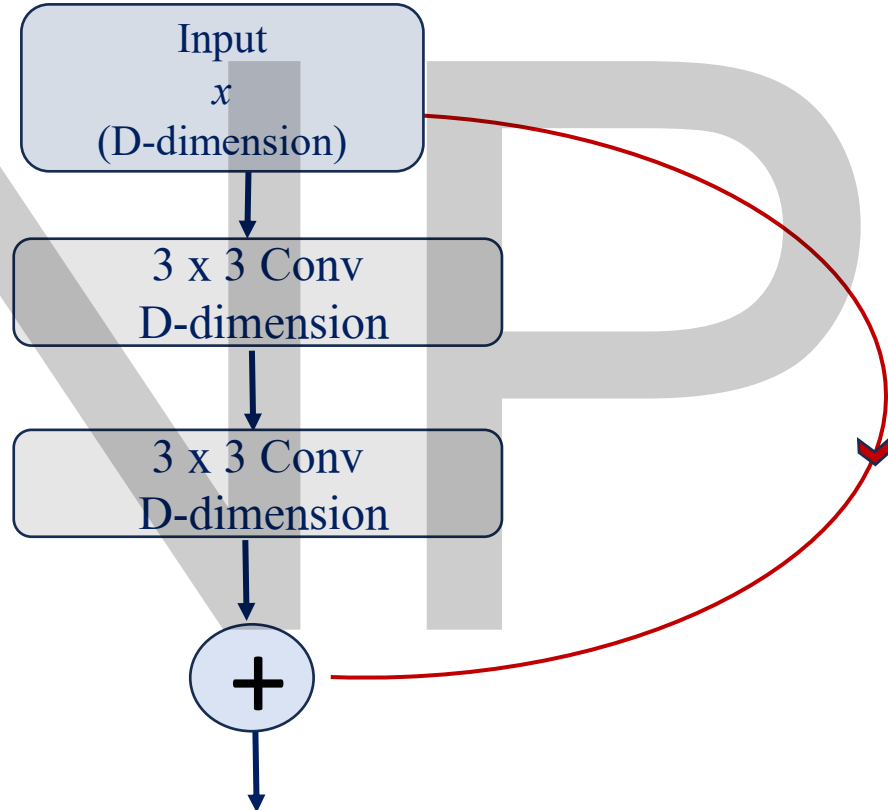
Even if $\frac{\partial F(x)}{\partial x}$ becomes small (due to small weights or saturating activations), the identity term I stops the gradient from vanishing completely.

Gradient can “flow” backward **unchanged** through many layers.

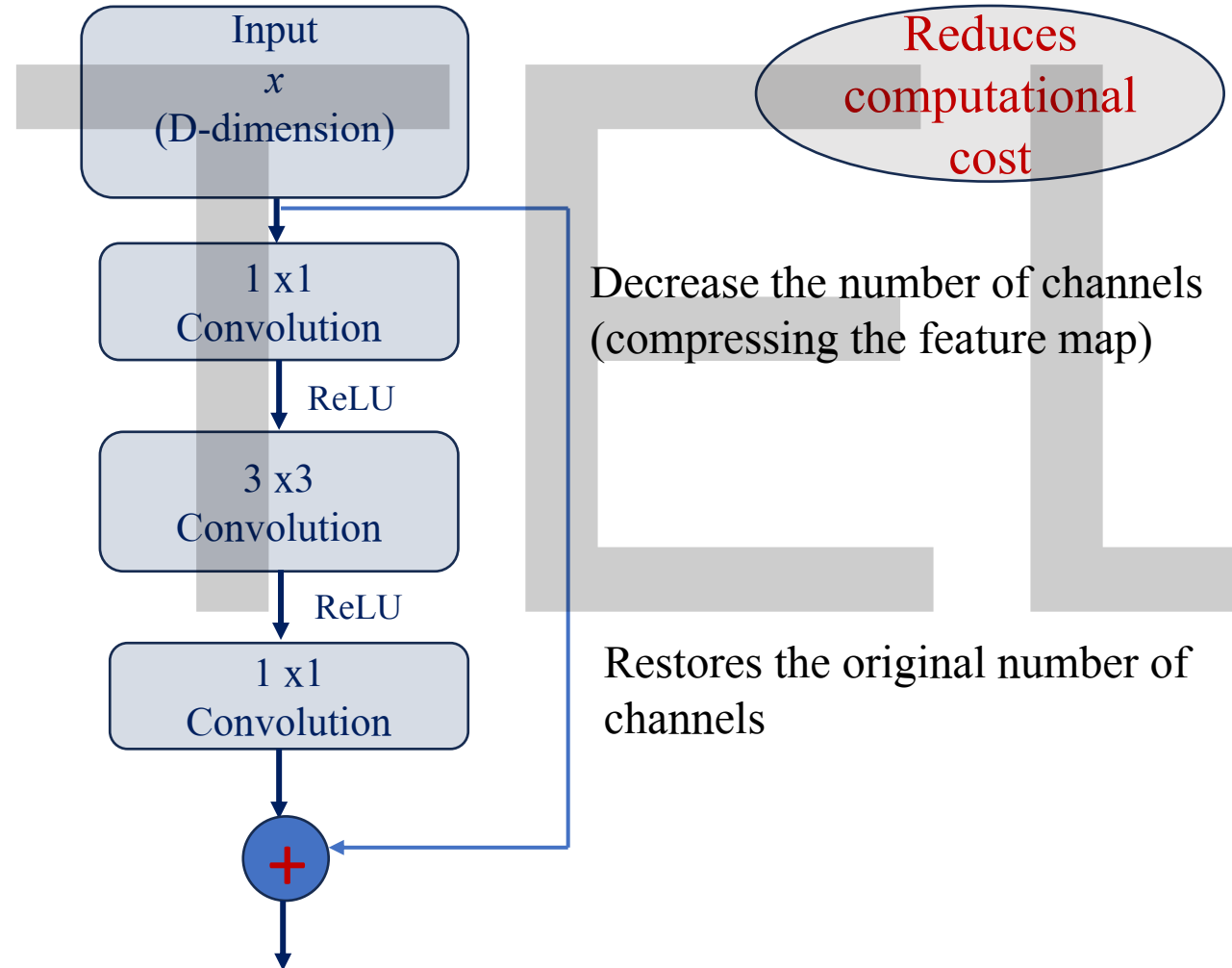


Variants of Residual Blocks

Basic Block



Bottleneck Block



Introduction to ResNet Model

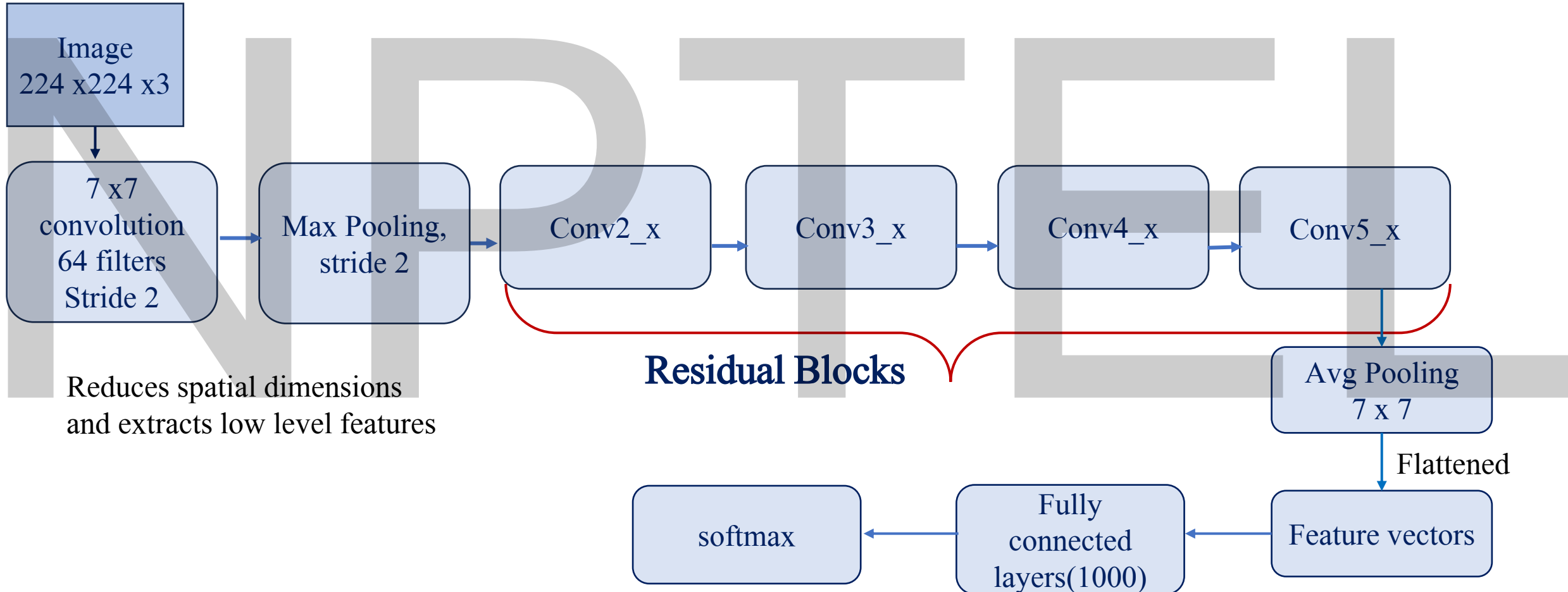
- Key idea –The architecture uses Residual learning to mitigate vanishing gradient problem
- Winner of ILSVRC 2015

Popular ResNet Architectures

- ResNet 34
- ResNet 50

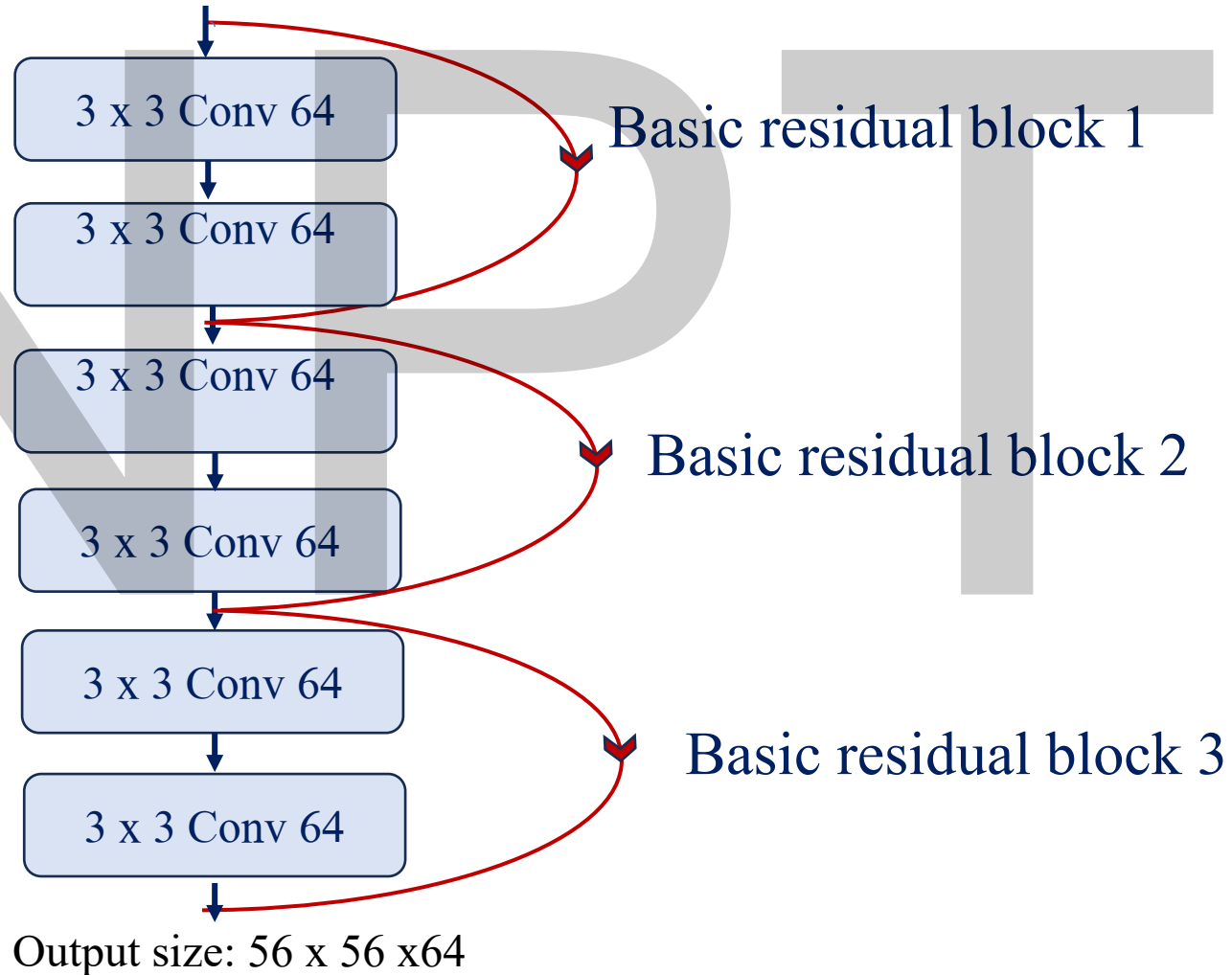
ResNet 34 Architecture

34-layer network architecture inspired by VGG-19 in which the shortcut/skip connections are added.



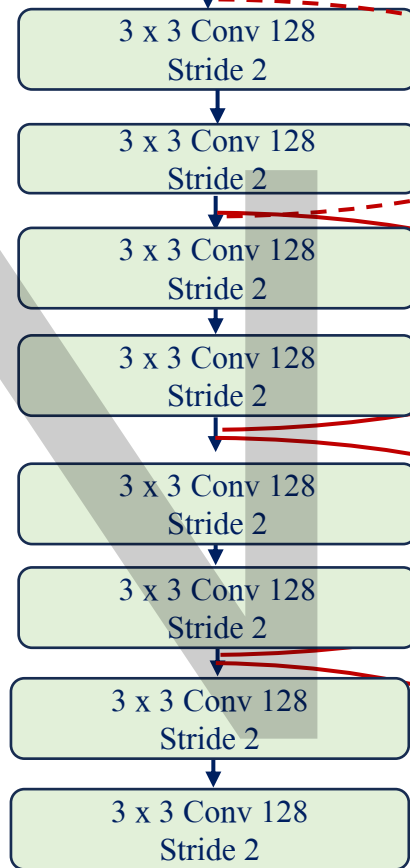
Architecture of Conv2_x Residual Block

Input size: 56 x 56 x64



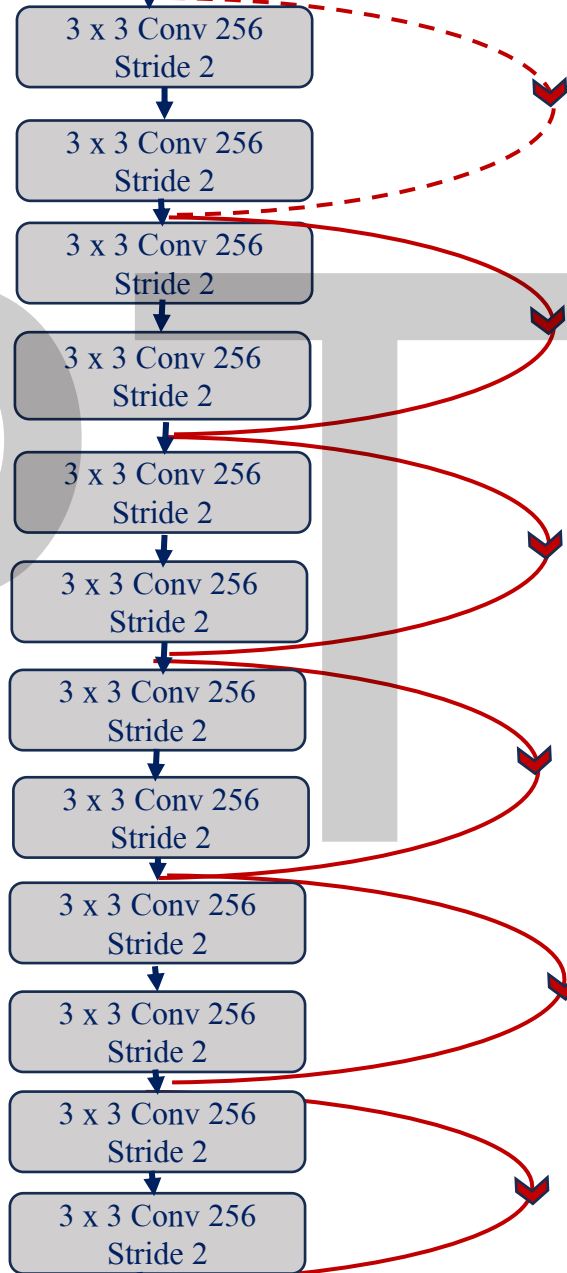
Conv3_x

Input size: 56 x 56 x 64

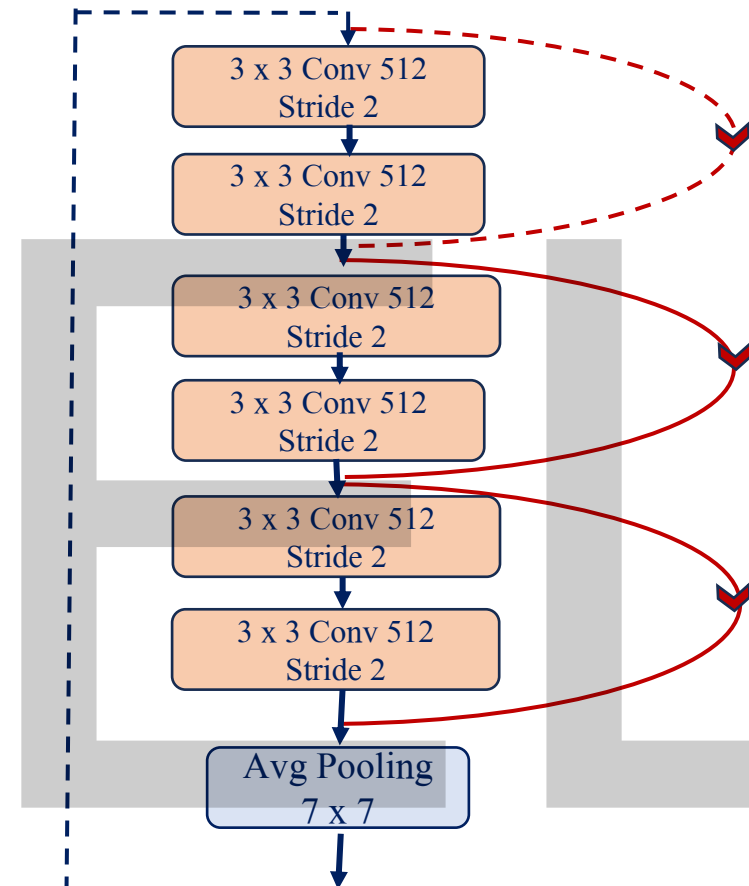


Output size:
56 x 56 x 128

Conv4_x



Conv5_x



Projection
Shortcut

Identity Shortcut

Identity shortcut and Projection shortcut

Identity Shortcut

Direct identity mapping: $H(x) = F(x) + x$

Used when the input x and the output of the main layer block $F(x)$ have the exact same dimensions

Projection Shortcut

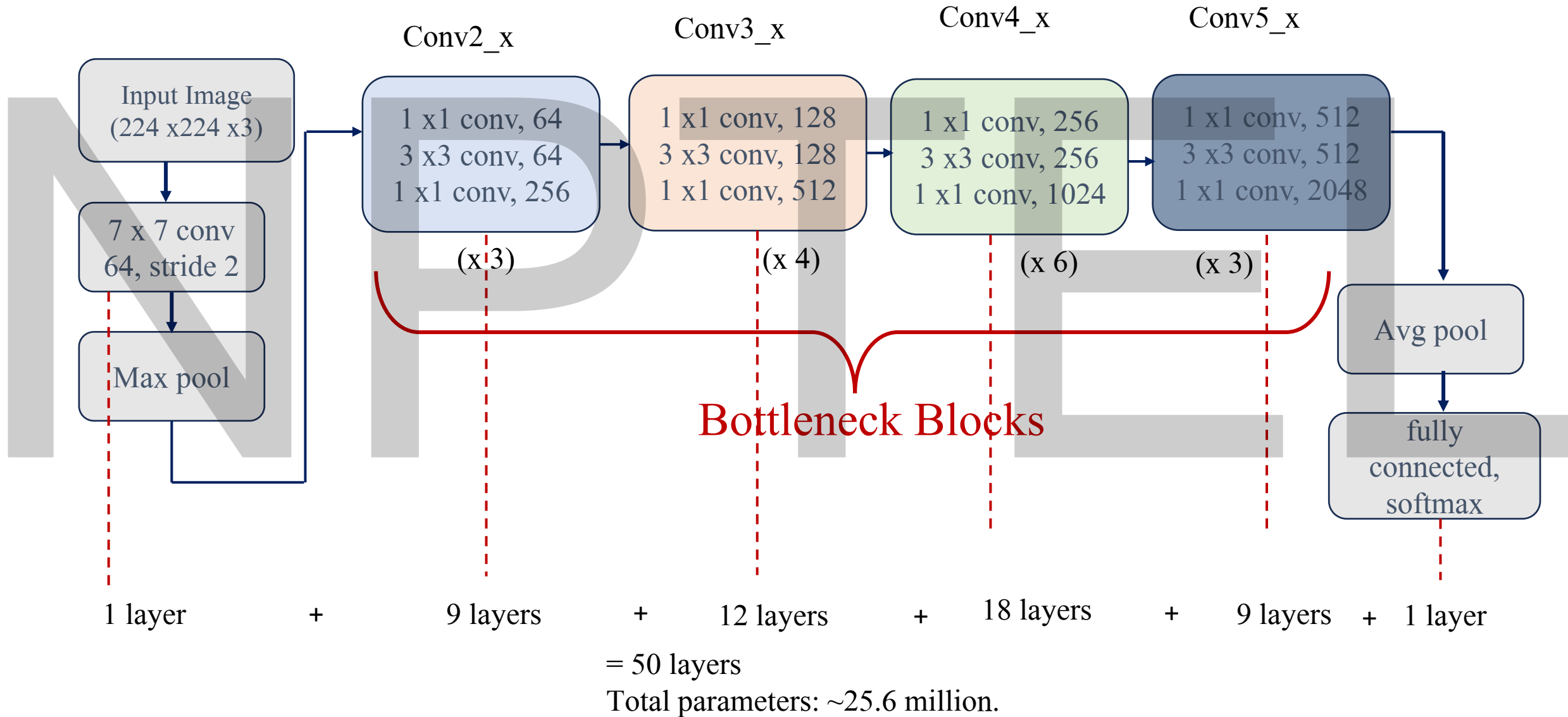
Requires modification of the input:
 $H(x) = F(x) + W_s x$

Used when the dimensions of the input x and the main layer output $F(x)$ do not match.

Input x must first pass through a **linear projection** W_s (typically a 1×1 convolution with stride) to resize it.

Element-wise addition operation can be performed.

ResNet50 Architecture



Conv2_x

1 x1 conv, 64
3 x3 conv, 64
1 x1 conv, 256

(x 3)

Conv2_1

(56 x56 x64)

1 x1 conv, 64
3 x3 conv, 64
1 x1 conv, 256

Conv2_2

1 x1 conv, 64
3 x3 conv, 64
1 x1 conv, 256

Conv2_3

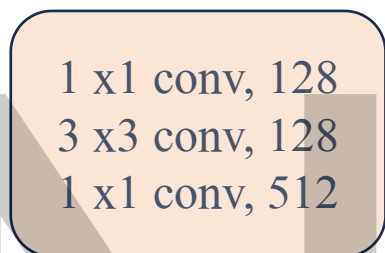
1 x1 conv, 64
3 x3 conv, 64
1 x1 conv, 256

Uses projection block

Uses Identity Block

Uses Identity Block

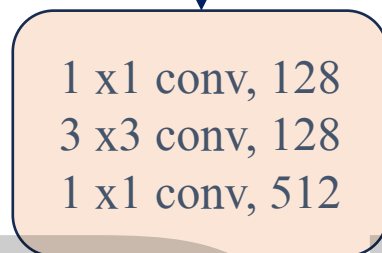
Conv3_x



(x 4)

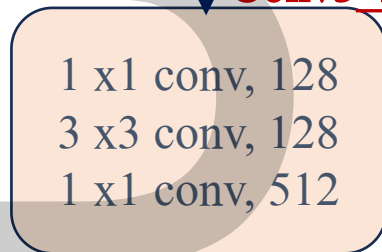
(56 x 56 x 128)

Conv3_1



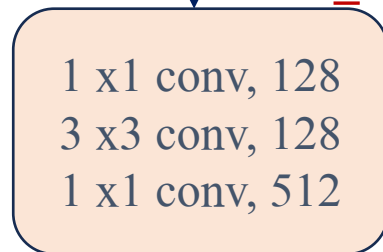
Uses projection block

Conv3_2



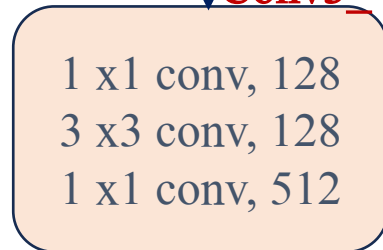
Uses Identity Block

Conv3_3



Uses Identity Block

Conv3_4



Uses Identity Block

Conv4_x

1 x1 conv, 256
3 x3 conv, 256
1 x1 conv, 1024

(x 6)

(56 x56 x 512)

Conv4_1

1 x1 conv, 256
3 x3 conv, 256
1 x1 conv, 1024

Uses projection
block

Conv4_2

1 x1 conv, 256
3 x3 conv, 256
1 x1 conv, 1024

Uses Identity
Block

Conv4_3

1 x1 conv, 256
3 x3 conv, 256
1 x1 conv, 1024

Uses Identity
Block

Conv4_4

1 x1 conv, 256
3 x3 conv, 256
1 x1 conv, 1024

Uses Identity
Block

Conv4_5

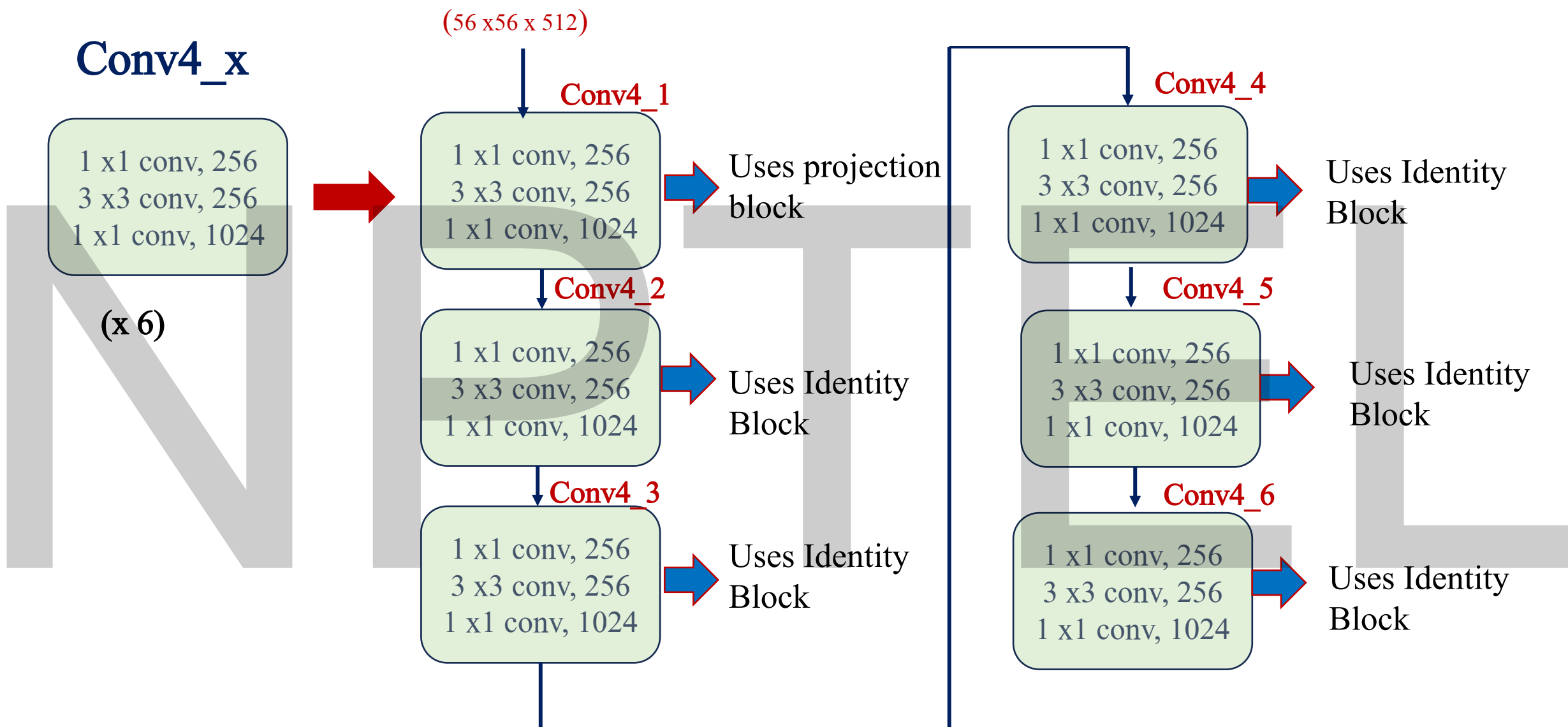
1 x1 conv, 256
3 x3 conv, 256
1 x1 conv, 1024

Uses Identity
Block

Conv4_6

1 x1 conv, 256
3 x3 conv, 256
1 x1 conv, 1024

Uses Identity
Block



Conv5_x

1 x1 conv, 512
3 x3 conv, 512
1 x1 conv, 2048

(x 3)

(56 x56 x 1024)

Conv5_1

1 x1 conv, 512
3 x3 conv, 512
1 x1 conv, 2048

Uses projection block

Conv5_2

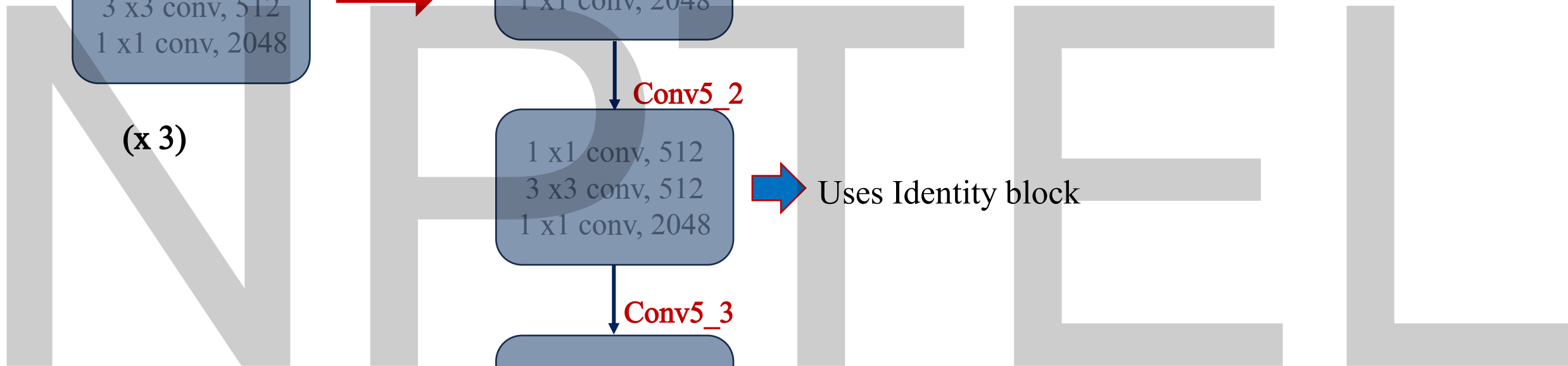
1 x1 conv, 512
3 x3 conv, 512
1 x1 conv, 2048

Uses Identity block

Conv5_3

1 x1 conv, 512
3 x3 conv, 512
1 x1 conv, 2048

Uses Identity block



Variants of ResNet Model

Model	Block Type	Depth	Typical Use Case
ResNet-18	Basic Block	18	Lightweight, small datasets
ResNet-34	Basic Block	34	Deeper, more accuracy than ResNet-18
ResNet-50	Bottleneck	50	Standard, balance of speed & accuracy
ResNet-101	Bottleneck	101	Higher accuracy, heavier
ResNet-152	Bottleneck	152	Very accurate, computationally expensive
ResNet-1202	Basic Block	1202	Research only, shows scaling limits

NPTEL

Summary

NPTEL

Thank you